

设计原理与框架

黄志安教授

电子邮箱: huang.za@cityu-dg.edu.cn

课程结构

- 假设J是项目所有功能的实现代码。
- ◆ 根据你的编程实践,你会将J放置在程序文件的哪个位置?
 - [假设]若J永远不会被修改或再次读取,是否需要考虑其位置?
 - ◆ 情景A: 将J置于main()函数主体中。
 - ◆ 方案B: 若关键在于将代码拆分至不同函数,为何不将J的单条语句分别放入各函数?
 - ◆ 场景C: 按功能划分J,将“子J”分别置于独立函数中,然后将它们粘合在一起。
 - ◆ 为何我们有时会“感觉”某个功能模块在某个程序中更易实现,而在另一个程序中却难以实现? 程序中实现比在另一个程序中更容易?

我们重视课程结构

David Parnas [10]

- ◆ 将同一程序分解为子程序的不同方式,在软件维护方面会产生截然不同的后果。
- ◆ 例如, X和Y是两个编程模块
 - 功能分解:
 - $P() = \{ x = X(w); y = Y(x); \dots W(x, y) \dots; \}$
 - $Y(x) = \{ \text{对数据结构 } x \text{ 执行任意操作} \}$
 - 面向对象分解:
 - z 是类 Z 的对象
 - $X(z) = \{ \text{对 } z \text{ 执行任意操作, 且 } z \text{ 保持完整性} \}$
 - $Y(z) = \{ \text{对 } z \text{ 执行任意操作, 且 } z \text{ 保持完整性} \}$
- $P = \{ \dots; X(z); \dots; Y(z); \dots W(z); \}$

当前: x 是列表变更: x 是图

影响所有模块

影响类 Z

程序结构的风格

- ◆ 更高层次上,我们涉及软件架构(或系统架构)、企业模式、架构模式
 - 软件应用程序组织的大局观
- ◆ 在更低层面上,我们关注编码惯例、编码原则、设计模式

源自何处

建筑始于？

伟大墙



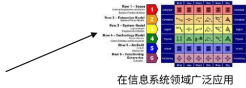
巨石阵



金字塔



文档



- ◆ 1992年：Perry & Wolf
- ◆ 1987年：J·A·扎克曼；1989年：M·肖
 - https://en.wikipedia.org/wiki/Zachman_Framework
- ◆ 1978/79：David Parnas，程序家族
 - 《关于将系统分解为模块的标准》
<http://dx.doi.org/10.1145/361598.361623>
- ◆ 1972年（1969年）：埃德斯杰·迪杰斯特拉，程序家族
- ◆ 1969年：I.P.夏普 @ 北约软件工程会议：

“我认为我们除了软件工程之外还有其他东西”。

- ◆ 1996年：开创性著作《软件架构视角下的新兴学科》
新兴学科的软件架构视角》

软件架构

- ◆ 基础在于模块化
 - 即：将模块分解为一组关联子模块的风格
 - 例如：计算机网络中OSI模型的分层风格
- ◆ 我们的目标是运用风格解决非功能性需求
- ◆ 例如：在初始化软件（功能需求）时如何处理“快速启动时间”（非功能需求）？
 - 将用户界面代码与程序状态初始化代码分离
 - 优先初始化影响UI的程序状态部分
 - 初始化质量较低但启动更快的核心模块，随后过渡到更高质量的完整核心。

软件架构

- ◆ 例如，在初始化软件（功能性要求）时，如何处理“快速启动时间”（非功能性要求）？
 - 将用户界面代码与程序状态初始化代码分离
- > 采用模型-视图-控制器模式，将给定模块分解为三个底层模块（模型、视图、控制器）
- 优先初始化影响UI的程序状态部分
- => 将模型模块划分为UI相关子模块、非UI相关子模块及维护模型完整性的子模块
- 初始化低质量、快速启动的核心模块，随后切换至高质量完整核心。
- => 模型层中的UI相关子模块应支持子模型间的可扩展性与兼容性。控制器应在不影响视图模块的前提下实现模型间的互操作性。

我们遵循的策略

- 在架构设计中，我们做到：
1. 递归地将模块分解为一组相互作用的子模块。为子模块分配功能责任，以满足“选定”的功能需求。
 - 例如：针对不同模型及模型完整性的子模块
 2. 为子模型分配非功能性质量属性
 - 例如：视图层的可用性、子模型层的可扩展性、控制器层的互操作性
 3. 为子模块分配非功能性约束
 - 例如：控制器不应使不同模型间发生互操作性冲突
- 影响视图模块

我们还观察到

- ◆ 在架构设计阶段，我们既不制定也不决定以下事项：
 - ◆ 功能需求的具体实现方式
 - 例如：无需考虑软件初始化算法或维护模型完整性的具体方法
 - ◆ 非功能性需求的具体实现方式
 - 我们仅为质量属性与非功能性约束分配说明/文档，以阐明它们为何对实现满足非功能性要求。

软件架构，定义

架构是系统在其环境中的基本概念或属性，体现在其元素、关系以及设计与演进原则之中。

(ISO/IEC/IEEE 42010 系统与软件工程
— 体系结构描述。(2011)

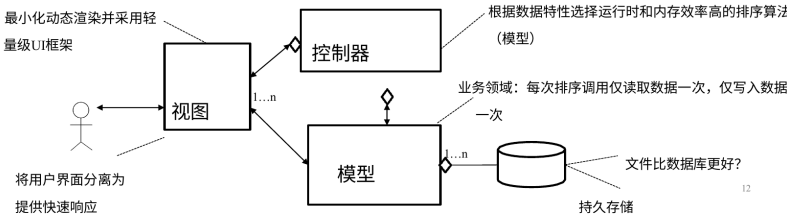
演化 = 维护

软件架构

- ◆ 架构是概念性的。
 - 例如，我们为不同模块赋予意义，使其交互过程阐释功能需求及其质量相关关注点的实现路径。
- ◆ 架构关乎根本性（重要）事项。
- ◆ 架构存在于特定上下文中。
 - 例如，若应用程序采用基于服务器的批处理模型，
 - 任何非凡的软件系统都具有架构。具体情境决定了该架构是否符合系统需求。

数据排序系统
(设计要求模糊)

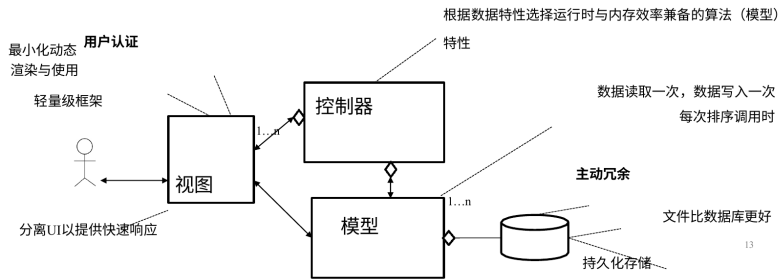
- ◆ 该系统应能
 - 快速响应用户的点击和按压
 - 在本地内存占用较小的情况下高效运行
 - 持久高效地保存排序结果



数据排序系统

(设计要求模糊)

- ◆ 设计过程中将识别更多设计要求
 - 更低层级，在用户界面内：高安全性 ⇒ 用户身份验证
 - 更低层级，数据层内部：可用性 ⇒ 主动冗余



13

一种数据排序系统

(附带“模糊”的设计要求)

- ◆ 我们可逐步探索更多“设计要求”，并优化系统以满足各项要求
- ◆ 例如：排序结果应正确
 - 策略A：使用大量测试用例验证系统
 - ▷ 项目本身应满足某些质量属性。或代码应具备可测试性，并在开发过程中进行充分测试 ▷ 可测试性
 - 策略B：部署后采用N版本编程的“控制器”
 - 实现N种不同排序算法，且控制器选择最一致的结果作为输出。

14

软件架构师如何看待自己的工作？

- ◆ 示例：
 - <https://medium.com/@nvashanin/the-path-to-becoming-a-software-architect-de53f1cb310a>
 - <https://www.jobhero.com/job-description/examples/computer-software/software-architect>
 - <https://www.redhat.com/architect/what-is-software-architect> (见下一张幻灯片)

15

软件架构师的技能组合

软件架构师



Software Architects: hard and soft skills needed to become a leader

Software Architects are part engineers, part business leaders. How much of each do you need to land the job? Here's what we found.

Posted February 26, 2023 | Matt Shady

Software Architect: hard skills needed

There's no one-size-fits-all definition because different projects may require different technical knowledge, but there are some skills that all projects will require.

- Unified Modeling Language (UML) is often listed as an essential requirement. What is certainly essential is familiarity with diagramming complex architectures.
- Searching for candidates with a deep knowledge of one or more programming languages essential to the business to support their long-term goals strategically. This could be anything: Java, JavaScript, Python, Ruby, Rust, Go, C, or even COBOL.
- Manage software development practices in a collaborative and agile fashion. That often means a deep familiarity with DevOps practices. This takes hard skills, such as creating an efficient DevOps environment, and soft skills, to keep development and operations teams aligned.

Software Architect: soft skills needed

However, just as important as the technology-related skills are the commonly called "soft" skills that can drive performance and bring team members together for successful projects.

- **Leadership** - Overseeing the development of a project and coordinating teams of developers to meet design standards requires significant leadership. Software Architects must be able to juggle the needs and demands of projects and teams.
- **Problem-Solving & Conflict Resolution** - Managing and coordinating all of the elements that go into a successful application project requires strong problem-solving skills - both technical and human.
- **Communication** - Communication is a key ingredient in any leadership position. To get the best of teams, Software Architects must clearly explain the mission, deadlines, and expectations.
- **Coaching & Inspiration** - If expectations aren't being met, leaders have to coach and inspire team members to achieve.
- **Organization** - Since Software Architects set the roadmap for development, being organized is key. Often large-scale and intricate UML diagrams are necessary, which requires a systematic and organized way of thinking.
- **Prioritizing** - Software Architects need to quickly prioritize tasks and juggle team members' assignments throughout a product's development.
- **Detailed Thinking** - In any development project, there are a significant number of details that must be managed correctly. This requires extreme attention to detail to make sure the project code meets objectives.
- **Creative Thinking** - The Software Architect has to move teams forward to accomplish a build regardless of the obstacles. This takes the ability to think creatively to find alternate solutions or creative ways to solve problems.

16

香港招聘职位

首席软件架构师

与amp;客户和团队

成员合作

技术主管

架构师

前端

和后端

开发人员

🕒 5 日前

🔒 全端

• Your Part.

• Live your passion for software development in exciting customer projects and be inspired by your colleagues' enthusiasm.

• As a Lead Software Architect, you will be responsible for developing IT architectures of innovative software solutions and providing strategic technology consulting for our customers.

Your Talent.

• After completing your studies in computer science or equivalent related field (bachelor's/master's), you've continued to develop and you can correctly evaluate new technologies and trends.

• Over several years in your profession, you have gained a broad knowledge of different technologies and methods.

• Your architectural decisions are characterised by non-functional requirements such as availability, flexibility, stability, ease of maintenance, and security while considering economic/business factors.

• On one hand, you are familiar with Enterprise Java technologies, web-based, and desktop clients, application servers, and relational and NoSQL databases. On the other hand, you know how systems outside the Java area are integrated and are familiar with technologies like REST, web services, and messaging.

• Expertise in front-end web development in JavaScript and experience with frameworks like ReactJS and Angular are a bonus!

• You have actively contributed to the career development and production environments.

• You are an experienced leader and are undoubtedly a huge reason for the success of adoption of agile in project teams. You have strong experience working closely with project management teams.

• You are aware of the concrete effects of your architectural decisions at the code level and you discuss them openly with developers.

• You have strong communication skills and you are sensitive to different requirements.

在香港刊登的职位

建筑

非功能性架构

敏捷流程

We are seeking a highly motivated and experienced AI Platform Architect to lead the design, development, and implementation of our cutting-edge AI platform. You will play a pivotal role in shaping the platform's architecture, ensuring its scalability, performance, and security. This is a unique opportunity to work on challenging technical problems and contribute to the advancement of our AI capabilities.

Responsibilities:

• Lead the overall architecture design of the AI platform, including requirement analysis, system design, technology selection, and technical detail management, ensuring a reasonable, efficient, and scalable architecture.

• Lead the AI platform development team, guide the design and development of core business modules, and address platform requirements and algorithm engineering challenges.

• Implement high-performance algorithm training and deployment resource acceleration through distributed computing, GPU acceleration, and other optimization techniques.

• Ensure the platform's scalability, high availability, and security, guaranteeing stable and efficient system operation.

• Stay abreast of the latest AI technologies and tools, continuously optimizing and refreshing the platform architecture.

• Undertake other tasks assigned by supervisors.

Qualifications:

• Bachelor's degree or above in Computer Science, Software Engineering, or a related field, five years of experience in platform development.

• Proven experience in designing and developing AI-related platform architectures.

• Strong understanding and hands-on experience with containerization technologies like Kubernetes and Docker.

• Familiarity with big data processing technologies such as Hadoop and Spark.

• Expertise in microservice architecture, distributed system design, and database technologies like MySQL and Redis.

• Proficient in Java and Python development with a solid foundation in Java technologies, including multithreading, NIO, collections, and memory tuning, demonstrated through practical project experience.

• Deep understanding and knowledge of the artificial intelligence field with a broad knowledge base. Experience in designing and implementing AI projects is a plus.

• Familiarity with Mlops frameworks and tools such as Git and Jenkins.

• Experience with DevOps/high development processes and project management.

• Experience with high-concurrency workloads is preferred.

• Proactive, highly responsible, quick learner, and adaptable to new working environments and challenges.

建筑师

◆ 负责系统的所有技术高层次设计决策

■ 高层次 = 架构层级

■ 当前所有决策或推迟至后续阶段的决策

■ 引导系统朝着正确方向演进

◆ 了解技术决策对业务的影响

■ 例如：对用户故事2做出决策1可能限制用户故事3的解决方案，这是否合理？

◆ 拥抱、预判并管理变更（及其风险）

◆ 作为项目愿景的倡导者与传播者

关注课程项目

◆ 在最终报告中记录架构决策

■ 在第一冲刺周期启动系统分解规划

■ 您所进行的系统分解；当前系统分解方案与您考虑过的其他系统分解方案之间的权衡取舍

■ 贵系统分解所采用的架构风格与模式

■ 例如：您的系统分解如何支持高“可扩展性”？是否采用了特定的架构风格或权衡点？

在利益相关方（包括开发商）之间

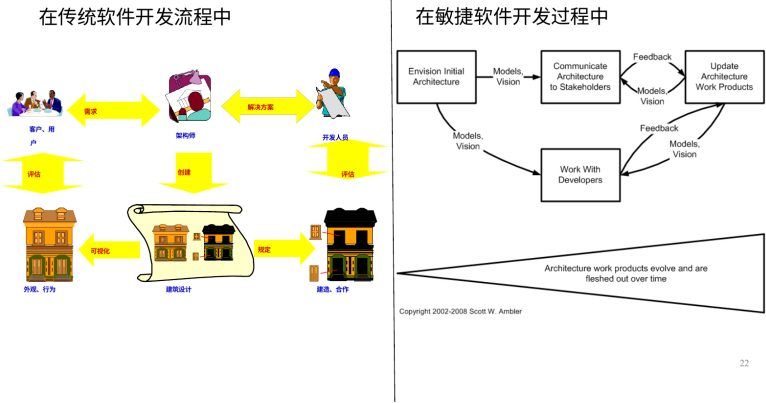
19

20

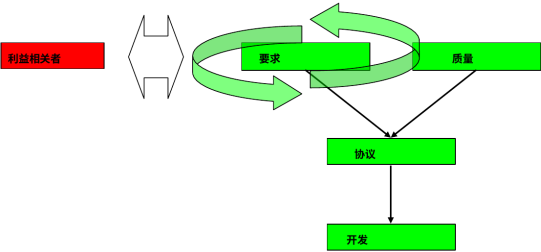
大纲

- ◆ 软件架构的作用
- ◆ 软件架构的驱动因素
- ◆ 质量属性与模式/策略
- ◆ 属性驱动设计
- ◆ 决策制定
- ◆ 架构风格

软件架构师的角色

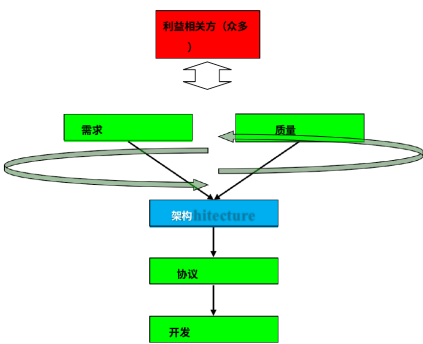


建筑前期生命周期



- 特性
- 主要围绕功能性要求进行迭代
 - 功能要求与质量要求之间不平衡

生命周期中的架构



- 特征
- 功能与质量要求的迭代

- 功能与质量要求的平衡

架构为何重要？

- ◆ 架构是利益相关者沟通的载体
- ◆ 架构体现**最早的设计决策集**
 - 对实现的约束
 - 决定组织结构
 - 抑制或启用质量属性

概述

- ◆ 软件架构的作用
- ◆ 软件架构的驱动因素

架构驱动因素

- ◆ 架构驱动因素是影响软件架构师**早期设计决策**的设计要求。
- ◆ 注：
 - 功能特性（即功能性）属于软件需求范畴，不足以作为架构驱动因素。
 - 架构处理的是非功能性需求（例如：实现相同功能但质量不同的解决方案）

新型基础设施的驱动因素示例

https://www.hkma.gov.hk/eng/key-functions/international-financial-centre/infrastructure/faster-payment-system.shtml

传统支付系统

- ◆ 功能
 - 账户间资金转移
 - 结算
- ◆ 优点
 - T+2结算
 - 单一平台，可靠
 - 商家和客户使用便捷

快速支付系统（FPS）

- ◆ 功能
 - 账户间资金转账
 - 结算
 - 二维码
- ◆ 优良品质
 - 实时且持续在线的资金转账与结算
 - 跨平台可靠
 - 商家与客户皆可轻松使用
 - 轻松添加新功能
 - 统一通用二维码标准

关于质量差异的观察

- ◆ 保持：可靠、商家和顾客易用
- ◆ 改进：T+2 → 实时且持续在线
- ◆ 通用化：单一平台→跨平台
- ◆ 扩展性：便捷新增功能，统一二维码标准
- ◆ 因此，即使二维码功能被禁用（导致传统支付系统与FPS在功能要求上无差异），FPS仍需解决其他质量属性

概述

- ◆ 软件架构的作用
- ◆ 软件架构的驱动因素
- ◆ 质量属性与模式/策略

功能需求

- ◆ 功能需求规定了软件系统为满足其存在的基本目的而需要完成的任务。

软件架构与质量

- ◆ 质量概念是软件架构的核心。
- ◆ 某些质量特性可通过运行观察：性能、安全性、可用性、功能性、易用性
- ◆ 另一些则无法通过运行观察：可修改性、可移植性、可复用性、可集成性、可测试性
- ◆ 质量同样影响着软件项目的各个方面

质量属性

- ◆ 质量属性是系统的可测量或可测试特性，用于表明系统满足利益相关者需求的程度
- ◆ 例如ISO/IEC 25010:2011标准中的质量属性模型



选读：简要阅读每个质量属性的阐述，以了解其概念
其内涵：<https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>

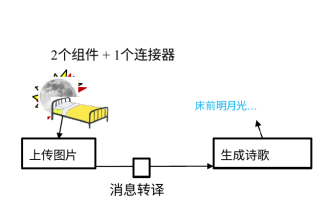
可测量/可测试

- ◆ 若不清楚当前系统距离目标的差距，就无法设计出满足该目标的系统。
 - 若理解“价格”的语境则可测试
 - [不可量化] 作为客户，我能看到所有加密货币的价格
 - [可量化] 作为客户，我能在一秒内查看所选20种加密货币的最新价格及4个历史价格。
 - 我们注意到，若将“一秒内”改为“0.02秒内”，系统需要更多设计才能实现这一目标。
- ◆ 作为分析师，我们应致力于获取可量化的指标。

质量属性权衡

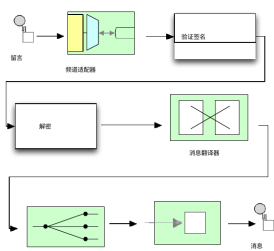
- ◆ 许多“性”与“性”的组合形成权衡关系。
 - 提升隐私性 → 降低可用性
 - 更高性能 → 更低互操作性
 - 更高模块化 => 更长上市周期
 - 更高可靠性 → 更低性能
 - 更快上市与更低成本 → 更低稳定性
 - ...
- ◆ 我们应当意识到，通过系统分解来解决某个质量属性时，应评估其对其他质量属性（在领域模型内及在需求质量属性中）

什么是系统分解？



质量示例：
快速上传；可扩展至大量用户

消息转发（隐藏复杂性）



设计品质示例

- ◆ 可行性
- ◆ 可复用性
- ◆ 一致性
- ◆ 简洁性
- ◆ 稳定性
- ◆ 组合性
- ◆ 可部署性

◆ 让我们快速了解一些流行的设计品质

可行性

- ◆ 项目成功的概率有多大？
- ◆ 上市时间（能否赶上D日？）
 - 更慢：从零开始构建；包含所有功能；自上而下的决策方式
 - 更快：尽可能复用；最小可行产品；开发者决策
- ◆ 经济可行性（资源是否充足以完成项目？）
 - 资金？
 - 资源？

- 人员（质量与数量）、时间、环境（含硬件）等
- 现有资源应对意外情况的弹性如何？

事件？

可复用性

- ◆ 复用策略
 - 复制（多份副本）与引用（共享副本）
 - 基线是项目内部还是外部？
 - 通用型、领域特定型还是功能特定型？
 - 更通用 => 对其他质量属性的约束更少
- ◆ 项目可用的复用前提条件 产品
 - 开源？行业标准？接口文档完善？组件可获取性？质量？

一致性

- ◆ 设计在概念上是否清晰连贯？
 - 理解部件和组件的一般原理有助于将理解推广到整个组件
 - 减少意外情况。
 - 例如：即使存在例外情况，也应尽可能在多个组件中统一处理，从而形成规范的异常处理机制。
- ◆ 遵循架构风格约束并记录主要架构决策
 - 反例：若模块化是约束条件，则禁止使用"跳转"指令在组件之间进行交互而不通过其接口
- ◆ 遵循设计原则，并以一致的理由打破它

简约

- ◆ 复杂的问题未必需要复杂的解决方案。
 - 简约即是美
 - 一个通用的解决方案胜过许多定制化方案
- ◆ 策略
 - 最小化组件变异性
 - 最小化组件间的直接连接
 - 确保每个设计决策简洁明了
 - 必要时重构设计
 - 识别基础原语集，并基于这些原语创建解决方案
- 需要牺牲简洁性的改变？ → 需三思

稳定性

- ◆ 架构设计是否稳定？
- ◆ 若某项设计决策依赖于 N 个模块，则修改其中一个模块可能导致其余 $N-1$ 个模块随之变更。
 - 这不是明智的设计决策。
 - 例如：当前项目使用100个开源组件。这些组件的兼容性问题以及不同的开发/修复速度，极易导致项目产品的不稳定性。
- ◆ 设计决策在特定运行环境中可能显得脆弱。
 - 建议将环境问题封装到底层平台或框架中
- ◆ 在缺乏充分测试的情况下使用实验性代码或功能评估就存在风险

可组合性

- ◆ 能否轻松地将辅助组件组合成更高阶的组件？组合程度如何？
 - 即插即用组件？
 - 是否完全自动化？
 - 通过修改映射脚本还是修改代码实现？
- ◆ 所有组件是否已准备就绪，还是需要开发某些“粘合”脚本？难度如何？

部署可行性

- ◆ 将系统部署到其运行环境是否容易？
- ◆ 手动部署 vs. 自动化部署
- ◆ 计划更新与持续交付
- ◆ 本地部署、云端部署还是服务组合？
- ◆ 版本降级：不可行 vs 可行（易/难）
- ◆ 是否需要协调外部依赖项？
- ◆ 升级补丁中的子集能否单独部署？
- ◆ 独立运行？

除设计质量外

- ◆ 质量还存在于其他维度：
 - 操作
 - 故障
 - 安全性
 - 需求或功能变更
 - ...

操作质量

- ◆ *性能*：延迟（响应时间、首次响应、完成时间）与吞吐量（单位时间输出量、带宽）
- ◆ *可扩展性*：随吞吐量（请求数）、用户数（并发量）、数据量（I/O与处理量）、网络规模（节点数）及系统规模（组件数）增长：
 - 是否支持动态扩展？
- ◆ *容量*：上限、过载及利用率
- ◆ *易用性*：易于学习且高效检索？符合规范？用户满意度？访问便捷性？
- ◆ *可维护性*：操作停止与连续运行？重启与否？手动错误报告与自动报告？
- ◆ *可见性*：系统行为可审计且有记录？

故障相关质量

- ◆ 可靠性
 - 平均故障间隔时间（MTBF）
 - 平均故障时间（MTTF）
- ◆ 可恢复性
 - MTTR：平均恢复时间、平均修复时间或平均响应时间
- ◆ (功能性)可用性
 - 可基于可靠性和可恢复性计算
 - 例如：可用性 = MTTF / (MTTF + MTTR)

安全相关质量

- ◆ 身份验证：确认用户身份
- ◆ 授权：限制访问权限
- ◆ 保密性：防止未经授权的数据访问
- ◆ 完整性：保护系统/数据免遭篡改
- ◆ 防御性：抵御攻击
- ◆ 隐私性：保障机密数据安全

变更相关质量

- ◆ 可配置性（推迟架构设计更佳）：
 - 如何初始化、(重新)激活、部署及竞标组件？
 - 如何增减系统资源（如节点或相同组件的冗余副本）
 - 功能如何启用或禁用？
 - 例如：运行时参数 > 启动时参数 > 硬编码参数
- ◆ 可定制性
 - 通用解决方案、产品线、UI皮肤、通过JSON数据解耦硬编码
- ◆ 持续时间
 - 临时性后撤销是否可行？永久性演变是否可行？
- ◆ 功能演进

变更相关质量

- ◆ 兼容性
 - 标准化接口、格式、平台。版本向后/向前兼容性
- ◆ 可移植性
 - 平台独立性与原生代码？写一次，到处运行？基于虚拟机？
- ◆ 互操作性（与其他系统）
 - 标准规范、调解器存在性、内容协商
- ◆ 集成
 - 标准接口与数据格式，持续性，实时性，数量
- 集成组件

项目健康质量

- ◆ 数据持久性、检查点与恢复
- ◆ 备份和灾难恢复计划
- ◆ 可维护性

例如：应用程序接口设计

- ◆ 在现代应用中，开发者常通过网络或库调用API。有时他们会将应用功能作为API暴露给其他客户端使用。
 - 例如，后端开发人员将他们的功能封装为API，供前端开发人员构建前端功能
- ◆ 我们希望API设计保持稳定、开放，并能为客户端应用提供支持
- ◆ 如何实现（从设计视角）？

例如：API设计原则

- ◆ 易于理解：易用性、简洁性、小巧性、完善文档、有意义的错误信息
- ◆ 服务质量高：可扩展性、可靠性、可用性、安全性、兼容性、标准化I/O消息类型
- ◆ 命名一致性（涵盖接口端点、函数、I/O、元素排序及与其他API的兼容性）
- ◆ 从客户端应用视角设计（易用性）
- ◆ 原则
 - 显式接口原则
 - 最小惊讶原则
 - 小接口原则
 - 统一访问原则
 - 少接口原则
 - 清晰接口原则

课后阅读：通过网络搜索了解这六项原则。

概要

- ◆ 软件架构的作用
- ◆ 软件架构的驱动因素
- ◆ 质量属性与模式/策略
- ◆ 属性驱动设计
 - 如您所知，我们在第6-13页的幻灯片中进行了说明。

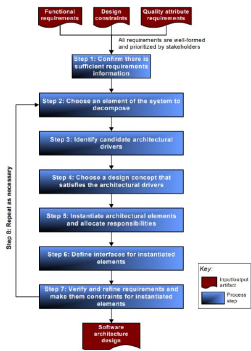
属性驱动设计（ADD）

◆ ADD是一种通过增量方式处理质量属性的架构设计方法论

- 1. 选择系统模块/连接器进行分解
- 2. 细化该模块：
 - 选择架构驱动因素（质量是核心驱动力）
 - 选择满足驱动因素的模式/策略/风格
 - 应用模式/策略/风格并分配职责
- 3. 重复步骤1-2

侧边栏：原始ADD模型

◆ 以下参考资料详细说明原始ADD的每个步骤
ADD的每个步骤。
https://resources.sei.cmu.edu/asset_files/TechnicalReport/2006_005_001_14795.pdf



示例 ADD 迭代

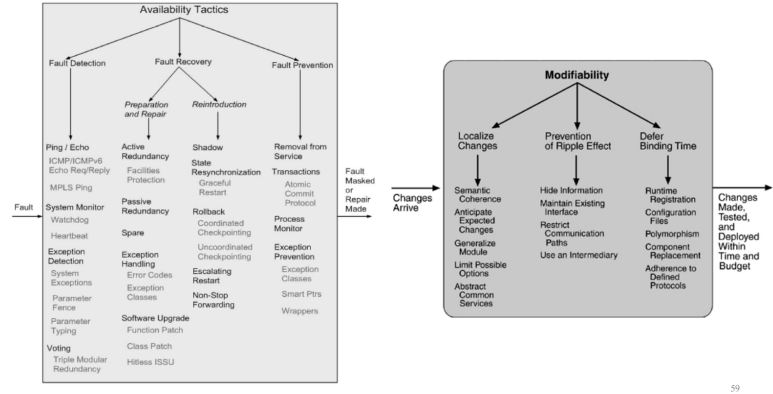
- ◆ 顶层（整个系统）：
 - 解决可用性问题
 - => 策略: 分离用户界面=> 模式: 三层架构（或如幻灯片12所示排序系统的MVC模式）
- ◆ 低层级，在展示层模块内：
 - 解决安全性
 - => 策略: 用户身份验证
- ◆ 更底层，数据层模块内：
 - 保障可用性
 - => 策略: 主动冗余

架构策略

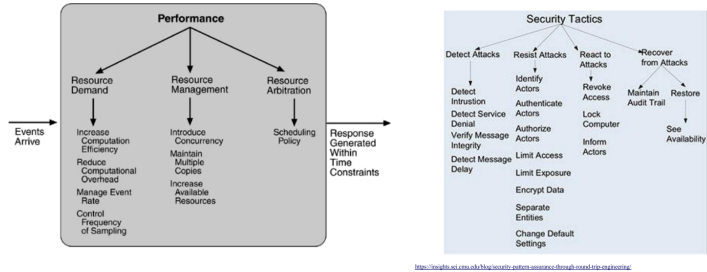
- ◆ ADD通过迭代应用一种或多种架构策略来解决质量属性问题。
- ◆ 架构策略是概念性方法，通常隐含于解决方案中。
- ◆ 当前最常提及的策略是安全策略（防御方法）。

可用性&可修改性

<https://insights.sei.cmu.edu/blog/tactics-and-patterns-for-software-robustness/>

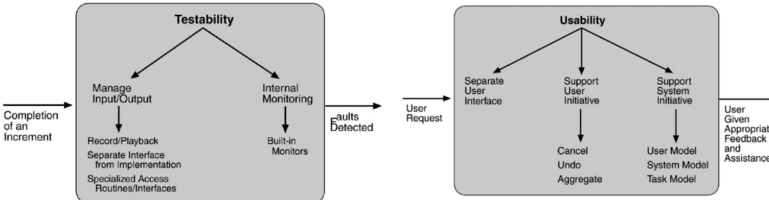


性能&安全性



<https://insights.sei.cmu.edu/blog/tactics-and-patterns-for-software-robustness/>

可测试性与可用性

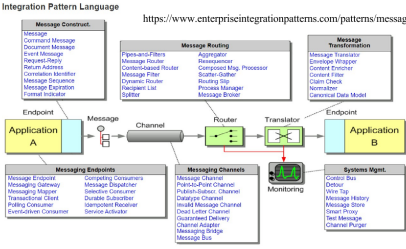


企业集成模式 [12]

- ◆ 除了通过组件来解决质量属性问题，我们还可以通过连接这些组件的方式来解决质量属性问题
- ◆ 接下来几张幻灯片将通过订单处理系统，结合若干企业集成模式[12]阐释这一理念。

企业集成模式 [12]

65种模式

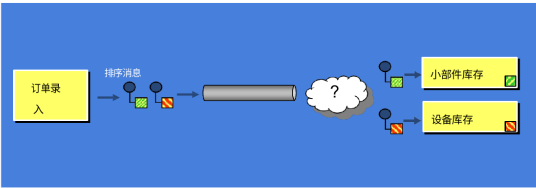


https://www.enterprisedintegrationpatterns.com/patterns/messaging/

- **Integration Styles** document different ways applications can be integrated, providing a historical account of integration technologies. All subsequent patterns follow the *Message* style.
- **Channel Patterns** describe how messages are transported across a *Message Channel*. These patterns are implemented by most commercial and open source messaging systems.
- **Message Construction Patterns** describe the intent, form and content of the messages that travel across the messaging system. The base pattern for this section is the *Message* pattern.
- **Routing Patterns** discuss how messages are routed from a sender to the correct receiver. Message routing patterns consume a message from one channel and republish it message, usually without modification, to another channel based on a set of conditions. The patterns presented in this section are specializations of the *Message Routing* pattern.
- **Transformation Patterns** change the content of a message, for example to accommodate different data formats used by the sending and the receiving system. Data may have to be added, taken away or existing data may have to be rearranged. The base pattern for this section is the *Message Transformation*.
- **Endpoint Patterns** describe how messaging system clients produce or consume messages.
- **System Management Patterns** describe the tools to keep a complex message-based system running, including dealing with error conditions, performance bottlenecks and changes in the participating systems.

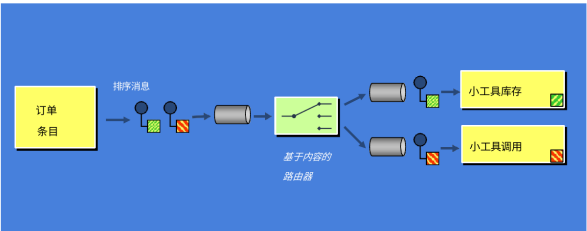
订单处理系统
问题1?

- ◆ 每个提供者只能处理特定类型的消息
- ◆ 将请求路由至"合适"的提供者。但如何实现？
 - 不希望让发送方承担决策负担
 - 允许供应商"挑选"消息需要协调



解决方案：
基于内容的路由器

- ◆ 插入一个根据消息元内容选择通道的路由器
- ◆ 路由器将传入消息转发至不同通道
- ◆ 消息内容不变
- ◆ 但若若干消息构成整体，则需进一步设计（详见下一页）



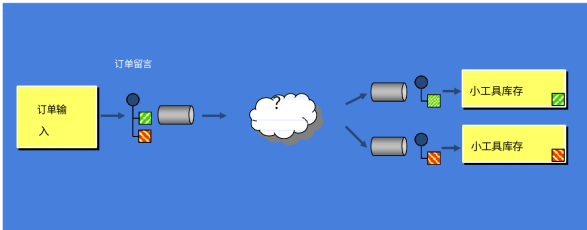
Apache Camel

```
RouteBuilder builder = new RouteBuilder() {
    public void configure() {
        errorHandler(deadLetterChannel("mock:error"));

        from("direct:in")
            .choice()
            .when(header("type").isEqualTo("widget"))
                .to("direct:widget")
            .when(header("type").isEqualTo("gadget"))
                .to("direct:gadget")
            .otherwise()
                .to("direct:other");
    }
};
```

问题2？

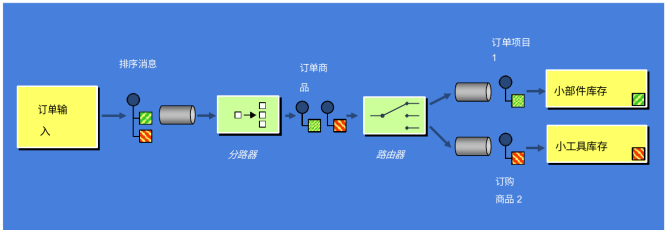
- ◆ 如何处理包含多个元素的消息？
 - 需要避免遗漏或重复元素
 - 高效利用网络资源



66

解决方案：拆分子器与路由器

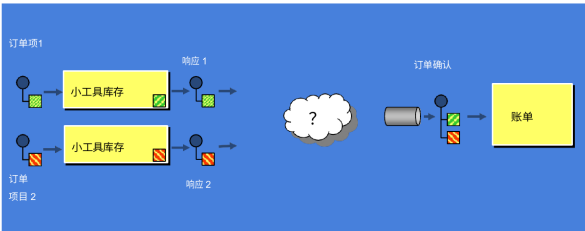
- ◆ 使用分路器将复合消息拆分为一系列独立消息
- ◆ 然后使用路由器按原有方式转发独立消息
- ◆ 请注意该方案包含两种模式组合
- ◆ 现在我们能拆分消息了，但反向操作呢...？



67

问题3a？ [单一伙伴]

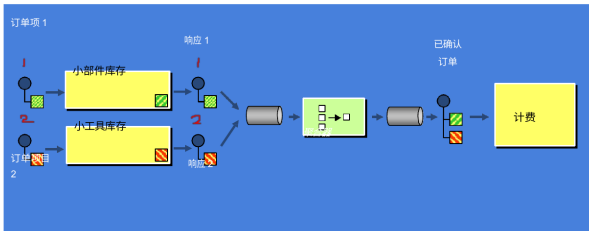
- ◆ 如何合并相关但独立消息的结果？
 - 消息可能出现乱序或延迟
 - 多个对话可能相互交织



68

解决方案：聚合器

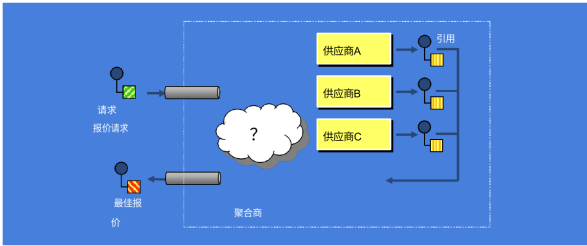
- ◆ 使用状态化过滤器（聚合器）
- ◆ 收集并存储消息直至接收完整数据集（完整性条件）
- ◆ 发布由单个消息聚合生成的单条消息（聚合算法）



69

问题3b: [多方参与者]

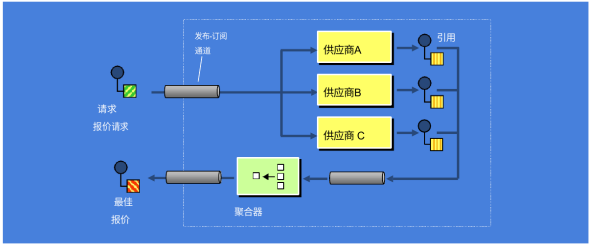
- ◆ 如何向动态收件人组发送消息？
- ◆ 并返回单一响应消息？



70

解决方案: 散射-聚合器对

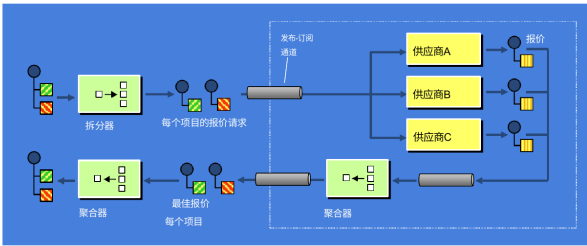
- ◆ 向发布-订阅通道发送消息
- ◆ 感兴趣的接收者订阅某个"主题"
- ◆ 聚合器收集单个响应消息
 - 可能不等待所有报价, 仅返回一个报价



71

复杂成分

- ◆ 接收订单消息
- ◆ 使用拆分为每个项目创建一条消息
- ◆ 发送至散发/聚合器, 该组件返回"最佳报价"消息
- ◆ 聚合生成报价订单消息



72

概述

- ◆ 软件架构的作用
- ◆ 软件架构的驱动因素
- ◆ 质量属性与模式/策略
- ◆ 属性驱动设计
- ◆ 决策制定

73

设计决策的应用

- ◆ 为利益相关者识别关键设计决策
 - 快速获取关键决策。例如向新人介绍并使其掌握最新信息。
 - ...获取决策依据，对照需求评估/验证决策
- ◆ 评估影响
 - 若需变更某元素，哪些关联要素将受影响（决策、设计、问题）？
 - 清理架构，识别关键架构驱动因素
- ◆ 明确设计决策
 - 例如：附带说明的架构描述
 - 例如，使用4+1架构视图来表示架构⁷⁴

表示法

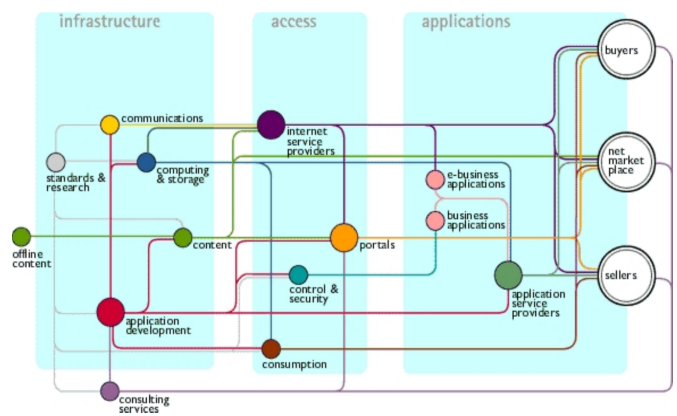
- ◆ 在制定设计决策时，我们需要将设计理念转化为**明确的成果（即表示形式）**，以便与利益相关者沟通
 - 例如：向团队成员及客户/用户展示非功能性需求相关说明

实践中的架构演示

- ◆ 主要分为两种形式：
 - PowerPoint幻灯片——面向管理者、用户、顾问等群体
 - UML图——面向技术人员
- ◆ 查看示例...请见下一页

该图展示的是管理员视角。
会计视角则截然不同，因其关注交易资金在系统中的流动与匹配情况。

示例

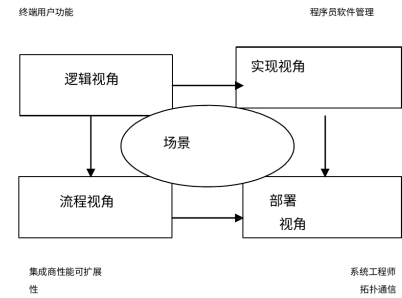


因此.....

- ◆ 不同的表现形式
 - 对于不同的人
 - 为不同的目的
- ◆ 这些表述既具有描述性又具有规范性

例如：4+1视图模型

[https://en.wikipedia.org/wiki/4%2B1_architectural_view_model]



UML中的4+1视图模型

- ◆ 逻辑视图
 - 类图、序列图、状态图
- ◆ 实现/开发视图
 - 组件图、包图
- ◆ 过程视图
 - 活动图
- ◆ 部署/物理视图
 - 部署图
- ◆ 场景

- 用例图

大纲

- ◆ 软件架构的作用
- ◆ 软件架构的驱动因素
- ◆ 质量属性与模式/策略
- ◆ 属性驱动设计
- ◆ 决策制定
- ◆ 架构风格

架构风格

- ◆ 架构风格是对**组件和连接器类型**及其运行时控制和/或数据传输模式的描述。
 - 尚未形成关于组件与连接器定义的正式概念
 - 通过实例学习
- ◆ 维基百科示例



- Blackboard
- Client-server (2-tier, 3-tier, n-tier, cloud computing exhibit this style)
- Component-based
- Data-centric
- Event-driven (or implicit invocation)
- Layered (or multilayered architecture)
- Microservices architecture
- Monolithic application
- Peer-to-peer (P2P)
- Pipes and filters
- Plug-ins
- Representational state transfer (REST)
- Rule-based
- Service-oriented
- Shared nothing architecture
- Space-based architecture

https://en.wikipedia.org/wiki/Software_architecture#Architectural_styles_and_patterns

基础架构风格类型[11]

Table 1: List of architectural style

Application Type	Architectural Style
Shared Memory	1. Blackboard 2. Data-centric 3. Rule-based
Distributed System	1. Client-server 2. Space based architecture 3. Peer-to-peer 4. Shared nothing architecture 5. Broker 6. Representational state transfer 7. Service-oriented
Messaging	1. Event-driven 2. Asynchronous messaging 3. Publish-subscribe
Structure	1. Component-based 2. Pipes and filters 3. Monolithic application based 4. Layered
Adaptable System	1. Plug-ins 2. Reflection 3. Microkernel

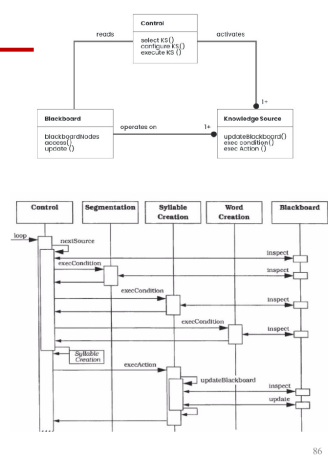
还有更多：例如模型-视图-控制器



还有更多：例如模型-视图-控制器

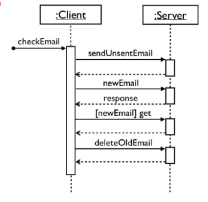
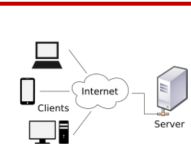
黑板

- ◆ 黑板 (BB)
 - 数据: 问题与部分解决方案。
 - ◆ 知识源 (KS)
 - 专用模块, 每个模块为整个问题的一部分提供解决方案。
 - Condition() 检查模块能否为黑板贡献内容
 - Acton() 更新黑板。
 - ◆ 控制
 - 定期监测BB上的变化
 - 在识别变更时, 决定KS应为
- 选定并执行。

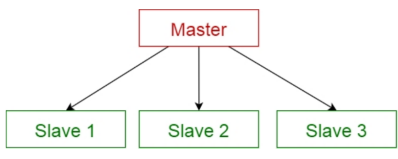
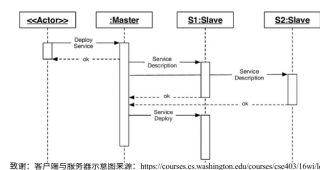


客户端-服务器及变体

◆ 客户端-服务器



◆ 主从

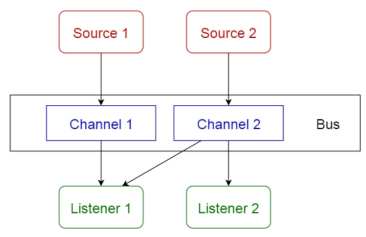


致谢: 客户端与服务器示意图来源: <https://courses.cs.washington.edu/courses/cse403/16wi/lectures/L10.pdf>

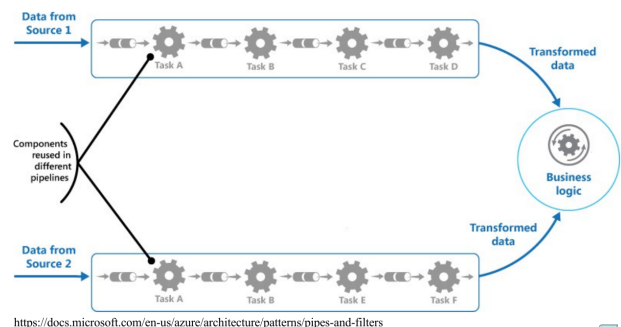
事件总线/发布-订阅模式

- ◆ 源节点向通道发布事件
- ◆ 通道被分组为总线
- ◆ 订阅者订阅特定通道

例如: Azure Service Bus队列



烟斗与过滤嘴

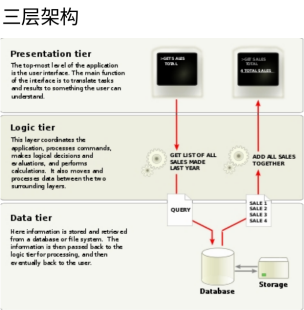
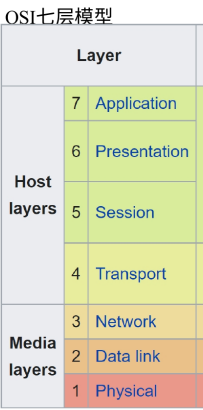


<https://docs.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters>

从源数据到转换后的数据, 数据经历一系列转换 (任务)。回顾编译器的功能分解图

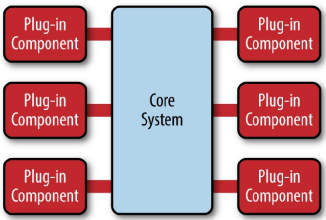


分层模式

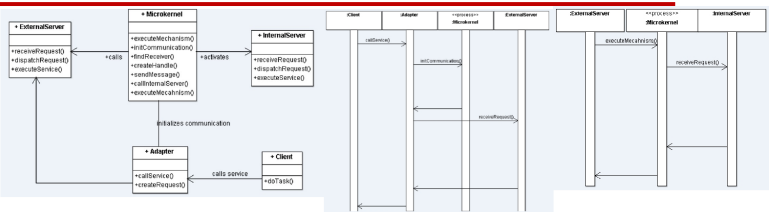


微内核

- ◆ 核心系统是精简系统
- ◆ 所有其他功能均以插件形式提供



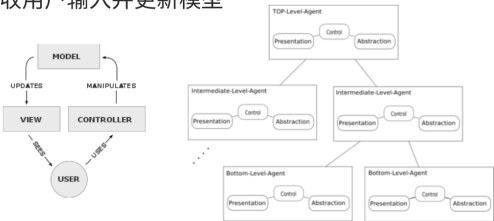
微内核



- 微内核提供核心功能集。
- 内部服务器为微内核增添了其他重要功能。
- 外部服务器 实现 更高级 功能 派生 自 微内核所提供的功能。
- 适配器在用户（客户端）与系统（微内核和外部服务器）之间架设桥梁，以处理异构系统组件。
- 课后阅读：阅读以下关于微内核：<https://www.guru99.com/microkernel-in-operating-systems.html>

模型-视图-控制器

- ◆ 流行度用于提升用户体验
- ◆ 模型：呈现核心功能与数据
- ◆ 视图层：向用户展示模型层的部分内容
- ◆ 控制器：接收用户输入并更新模型



概要

- ◆ 软件架构存在于特定上下文中（即在部署阶段交付功能需求时的质量属性）
- ◆ 我们通过风格将系统划分为模块及其连接关系以满足质量属性（例如通过ADD）
- ◆ 存在多种架构风格
 - 成功的架构被归纳为模式
- ◆ 架构描述包含利益相关者的关注点及所选架构的依据，以视图和视角的形式呈现。

参考文献与资源

1. Rick Kazman, Paul Clements, Len Bass. 《软件架构实践》第三版。
 - 第二版参见：<http://www.ece.ubc.ca/~matei/EECE417/BASS/>
2. 4+1视图模型 https://en.wikipedia.org/wiki/4%2B1_architectural_view_model
3. 微服务 <http://microservices.io>
4. 微前端 <https://micro-frontends.org>
5. 无服务架构 <https://martinfowler.com/articles/serverless.html>
6. ISO/IEC/IEEE 42010 系统和软件工程——体系结构描述 <https://ieeexplore.ieee.org/document/6129467>
7. 汉斯·范·维莱特 https://en.wikipedia.org/wiki/Hans_van_Vliet
8. 新兴模式的概述、优缺点 <https://www.slideshare.net/VincentComposieux/architecture-81270165>
9. 软件架构设计游戏：<http://smartdecisionsgame.com/>
10. E.W. Dijkstra, *Communications of the ACM*. 11 (3): 147–148, 1968. doi:10.1145/362929.362947
11. 软件架构风格与模式的完整综述，<https://doi.org/10.1016/j.procs.2015.10.019>
12. 企业集成 模式，<https://www.enterpriseintegrationpatterns.com/patterns/messaging/>
13. 《软件架构基础：工程方法论》。O'Reilly Media出版社。